

SAPIENZA
UNIVERSITÀ DI ROMA

Distributed Systems

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Alessandro Mei

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

1 Sistemi distribuiti e computazioni **1**
1.1 Il problema: valutare un predicato globale 1
1.2 Il modello: sistemi distribuiti asincroni 3
1.3 Computazioni distribuite 5

2 Stati globali, tagli e consistenza **8**
2.1 Stati globali, tagli e run 8
2.2 Monitoraggio attivo e deadlock fantasma 10
2.3 Tagli e run consistenti 11
2.4 Il reticolo degli stati globali 12

3 Osservare una computazione: orologio reale e clock logici **16**
3.1 Monitoraggio passivo e osservazioni 16
3.2 Regole di consegna e consegna FIFO 17
3.3 Orologio reale e clock condition 17
3.4 Clock logici di Lamport 18
3.5 Consegnare le notifiche in ordine di clock logico 20

4 Consegna causale e vector clock **21**
4.1 Consegna causale 21
4.2 La strong clock condition 22
4.3 Storie causali 23
4.4 Vector clock 24
4.5 Confrontare i vector clock 25
4.6 Consegna causale con i vector clock 27

5 Snapshot distribuiti **30**
5.1 Stati dei canali 30
5.2 Primo protocollo: un orologio reale 31
5.3 Dall'orologio reale al protocollo di Chandy–Lamport 31
5.4 Il protocollo di Chandy–Lamport 33
5.5 Perché funziona, e perché servono i canali FIFO 34
5.6 Proprietà dello snapshot 34

Capitolo 1

Sistemi distribuiti e computazioni

Molti problemi dei sistemi distribuiti hanno la stessa forma: bisogna accorgersi che lo stato complessivo del sistema soddisfa una certa condizione e, quando succede, reagire. Per affrontarli serve prima un modello preciso di che cosa sia un sistema distribuito e di come si svolga una computazione al suo interno. Questo capitolo fissa il modello usato nei capitoli seguenti: processi che comunicano solo con messaggi, senza un orologio comune, e un ordine tra gli eventi basato soltanto su cause ed effetti.

1.1 Il problema: valutare un predicato globale

Ecco alcuni problemi tipici dei sistemi distribuiti:

- rilevare un *deadlock*, cioè un gruppo di processi bloccati, ciascuno in attesa di un altro;
- capire se una computazione distribuita è terminata;
- accorgersi che il *token* di un protocollo a gettone è andato perso;
- trovare gli oggetti che nessun processo può più raggiungere, per liberarne la memoria (*garbage collection* distribuita);
- salvare uno stato da cui ripartire dopo un guasto (*checkpointing*);
- riconfigurare un programma mentre è in esecuzione, per esempio per bilanciare il carico;
- fare *debugging* e monitoraggio di un'applicazione distribuita.

In tutti questi casi bisogna controllare una condizione sullo stato complessivo del sistema: i processi sono bloccati in un ciclo di attese? La computazione è finita? Il *token* esiste ancora? Una condizione così si esprime con un predicato.

Un **predicato** è un'affermazione sullo stato del sistema che può essere vera o falsa, come «la computazione è finita». **Valutarlo** vuol dire stabilire se, in un certo stato, è vera o falsa. Quanto sia facile farlo dipende da quali informazioni servono. Un **predicato locale** dipende solo dallo stato di un processo: «p_1 sta aspettando

una risposta» è vero o falso a seconda delle variabili di p_1, e p_1 lo può valutare da solo. Un **predicato globale** dipende invece dagli stati di più processi insieme, e spesso anche dai messaggi in viaggio tra loro: nessun processo lo può valutare da solo, perché ognuno conosce soltanto il proprio stato.

Le condizioni dei problemi elencati sono predicati globali, che si indicano con Φ (la lettera greca *phi* maiuscola). Per esempio:

- *deadlock*: Φ = «c'è un ciclo di processi, ciascuno in attesa del successivo». p_1 sa di aspettare p_2, ma non sa se p_2 aspetta p_3 e p_3 aspetta p_1: il ciclo si vede solo mettendo insieme gli stati di tutti;
- terminazione: Φ = «tutti i processi sono inattivi e non ci sono messaggi in viaggio». Non basta che ogni processo sia inattivo, perché un processo inattivo che riceve un messaggio si rimette al lavoro;
- *token* perso: Φ = «nessun processo ha il *token* e nessun messaggio in viaggio lo trasporta».

Definizione (Global Predicate Evaluation). La **Global Predicate Evaluation** (GPE, valutazione di un predicato globale) è il problema di stabilire se lo **stato globale** del sistema, cioè l'insieme degli stati dei singoli processi, soddisfa un predicato globale Φ.

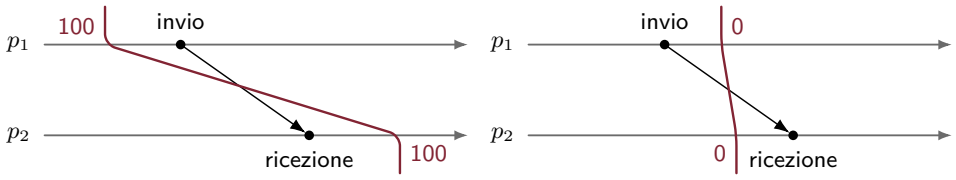
Se ne occupa un processo, indicato con p_0, che dal capitolo 2 si chiamerà monitor. GPE è un problema generale perché non dipende da quale Φ si sceglie: un metodo che lo risolve è il cuore della soluzione di tutti i problemi elencati, e per ciascuno resta da scegliere il predicato giusto e la reazione da eseguire quando è vero, per esempio interrompere un processo del ciclo o rigenerare il *token*.

Su un solo computer valutare un predicato è facile: le variabili da cui dipende stanno nella stessa memoria e si leggono tutte nello stesso momento. In un sistema distribuito i processi non condividono memoria e comunicano solo scambiandosi messaggi. Un processo che vuole conoscere lo stato globale deve farsi mandare gli stati degli altri, e i messaggi impiegano tempi variabili e imprevedibili: ogni stato viene letto in un momento diverso, e intanto il sistema va avanti.

Esempio (un bonifico). Due processi gestiscono due conti bancari: p_1 ha 100 euro e p_2 nessuno. p_1 fa a p_2 un bonifico di 100 euro: toglie la somma dal proprio saldo e la invia con un messaggio, e p_2 la aggiunge al suo quando il messaggio arriva. Il denaro non si crea e non si distrugge, quindi i due saldi, più il denaro in viaggio, fanno sempre 100 euro. Il processo p_0 controlla che sia così e valuta Φ = «il denaro complessivo non è 100 euro», cioè «c'è un errore»: chiede il saldo a p_1 e a p_2 e somma le risposte. Le richieste arrivano ai due processi in momenti diversi, e la Figura 1.1 mostra due casi.

(a) La richiesta arriva a p_1 *prima* che invii il bonifico, e p_1 risponde 100; arriva a p_2 *dopo* che l'ha ricevuto, e anche p_2 risponde 100. La somma è 200 euro e p_0 segnala un errore che non c'è mai stato. Lo stato ricostruito è impossibile: p_2 ha ricevuto un bonifico che p_1 non ha ancora inviato.

(b) La richiesta arriva a p_1 *dopo* l'invio, e p_1 risponde 0; arriva a p_2 *prima* della ricezione, e p_2 risponde 0. Anche qui p_0 segnala un errore, ma per un motivo diverso: lo stato è possibile, e i 100 euro sono nel messaggio in viaggio, che nessuna delle due risposte comprende. Per non sbagliare p_0 deve contare anche i messaggi in viaggio, per esempio facendosi dire da p_1 i bonifici inviati e da p_2 quelli ricevuti.



(a) 100 + 100 = 200 (b) 0 + 0 = 0, e 100 in viaggio

Figura 1.1. Il bonifico dell'esempio: il tempo scorre da sinistra a destra, e la freccia è il messaggio con i 100 euro. Le linee rosse passano per i punti in cui le richieste di p_0 arrivano a p_1 e a p_2; accanto, in rosso, i saldi comunicati.

In generale lo stato che un processo ricostruisce in questo modo può essere:

- *obsoleto*: quando le risposte arrivano, il sistema è già andato avanti, e nel caso (b) il bonifico potrebbe essere già arrivato;
- *incompleto*: alcune informazioni sono ancora in viaggio, come i 100 euro nel caso (b);
- *inconsistente*: mette insieme pezzi presi in momenti incompatibili e descrive una situazione in cui il sistema non si sarebbe mai potuto trovare, come i 200 euro nel caso (a).

Comunicare più spesso rende la vista più aggiornata e più completa, ma non basta a renderla consistente: bisogna ragionare sull'ordine in cui le informazioni vengono osservate e su che cosa contengono. Risolvere GPE vuol dire quindi costruire uno stato globale *consistente*, cioè uno in cui il sistema si sarebbe potuto trovare davvero, e valutare Φ su quello. Il capitolo 2 definisce con precisione la consistenza. I capitoli successivi mostrano due modi di costruire stati globali consistenti: in uno un processo *osserva* passivamente il sistema (capitoli 3 e 4), nell'altro lo *interroga* attivamente (capitolo 5).

1.2 Il modello: sistemi distribuiti asincroni

Definizione (sistema distribuito). Un **sistema distribuito** è un insieme di processi sequenziali $p_1, \ldots, p_n$ e una rete che realizza **canali** di comunicazione unidirezionali tra coppie di processi. Nel nostro modello:

- i canali sono **affidabili**: ogni messaggio inviato prima o poi arriva;
- i canali **non sono FIFO**: possono consegnare i messaggi in un ordine diverso da quello di invio;

- la rete è **fortemente connessa**: ogni processo può comunicare con ogni altro, eventualmente passando per processi intermedi, anche se non c'è un canale diretto tra ogni coppia.

Quando si sceglie un modello conviene fare meno ipotesi possibile. Le due grandezze su cui si può contare o no sono la velocità relativa dei processi e il ritardo dei messaggi.

Definizione (sistema asincrono). Un sistema distribuito è **asincrono** se non esiste alcun limite né alle velocità relative dei processi né ai ritardi dei messaggi. Se invece questi limiti esistono e sono noti, il sistema è **sincrono**.

In un sistema asincrono un processo non può contare su orologi sincronizzati con quelli degli altri, né ragionare in termini di un tempo reale comune: l'unico strumento di sincronizzazione è la comunicazione. Ci sono due buoni motivi per studiare proprio questo modello.

- *È il più debole.* Fa meno ipotesi di qualunque altro, quindi una soluzione che funziona in un sistema asincrono funziona in qualunque sistema distribuito. Il suo costo è anche un limite superiore: in ogni altro sistema lo stesso problema si risolve con un costo non maggiore.
- *È realistico.* I componenti fisici sono spesso sincroni (le velocità dei processori e i ritardi sui collegamenti sono limitati), ma gli strati di software che li condividono tra molti processi, come *scheduling*, code e ritrasmissioni, rendono imprevedibili i tempi visti dalle applicazioni. Il sistema che ne risulta si descrive meglio come asincrono.

Non vale invece il contrario. Un algoritmo pensato per un sistema sincrono può sfruttare i limiti di tempo, per esempio concludendo che un processo non ha inviato nulla perché entro un certo tempo non è arrivato niente; in un sistema asincrono questo ragionamento è sbagliato, perché un ritardo lungo è sempre possibile.

Il teorema CAP. Un risultato classico sui sistemi che conservano dati replicati su più nodi è il **teorema CAP**. Riguarda tre proprietà:

- *Consistency* (C): tutti i nodi vedono gli stessi dati, come se ne esistesse una sola copia, e una lettura restituisce sempre l'ultimo valore scritto;
- *Availability* (A): ogni richiesta a un nodo funzionante riceve una risposta;
- *Partition tolerance* (P): il sistema continua a funzionare anche se la rete si divide in parti che non riescono a comunicare tra loro.

Il teorema afferma che un sistema distribuito non può garantirle tutte e tre insieme, ma al massimo due. Poiché in una rete reale le partizioni possono sempre capitare, in pratica la scelta è tra consistenza e disponibilità durante una partizione. Attenzione al nome: la *consistency* di CAP riguarda le copie dei dati e non c'entra con la consistenza di tagli e stati globali del capitolo 2.

1.3 Computazioni distribuite

Una **computazione distribuita** è l'esecuzione di un programma distribuito da parte dei processi del sistema. L'attività di ogni processo, che è sequenziale, si descrive come una sequenza di **eventi**. Un evento può essere:

- **interno**: cambia soltanto lo stato locale del processo;
- un **invio** $send(m)$: il processo mette il messaggio m sul canale verso il destinatario;
- una **ricezione** $receive(m)$: il processo preleva m dal canale in arrivo. Perché avvenga, m deve essere arrivato e il processo deve essere pronto a riceverlo; altrimenti uno dei due aspetta l'altro.

Supponiamo che ogni messaggio abbia un identificatore unico (basta aggiungere al contenuto l'indice del mittente e un numero di sequenza): così ogni ricezione corrisponde esattamente a un invio, anche se più processi inviano lo stesso dato. A questo livello di astrazione qualunque forma di comunicazione di più alto livello, come chiamate di procedura remota, *broadcast*, transazioni distribuite o memoria condivisa distribuita, si riduce a coppie di eventi di invio e ricezione.

Definizione (storia locale e storia globale). La **storia locale** di p_i è la sequenza, eventualmente infinita, dei suoi eventi

$$h_i = e_i^1 \, e_i^2 \, e_i^3 \ldots$$

numerati nell'ordine in cui p_i li esegue: è l'**enumerazione canonica**. Con $h_i^k = e_i^1 \ldots e_i^k$ si indica il prefisso dei primi k eventi (h_i^0 è la sequenza vuota). La **storia globale** è l'insieme di tutti gli eventi della computazione: $H = h_1 \cup \cdots \cup h_n$.

1.3.1 Precedenza causale

La storia globale dice quali eventi sono avvenuti, ma non *quando* gli eventi di processi diversi sono avvenuti l'uno rispetto all'altro: in un sistema asincrono non c'è un tempo comune a cui riferirsi. L'unico ordine che ha senso è quello di causa ed effetto: un evento deve venire prima di un altro solo se può averlo influenzato, cioè se c'è stato un flusso di informazione dal primo al secondo. L'informazione può passare in due modi: tra due eventi dello stesso processo, che lavorano sullo stesso stato locale, oppure dall'invio alla ricezione di uno stesso messaggio.

Definizione (precedenza causale). La **precedenza causale** $\rightarrow$ (*causal precedence*) è la relazione tra gli eventi di H definita da tre regole:

1. se e ed e' sono eventi dello stesso processo ed e viene prima di e', allora $e \rightarrow e'$;
2. se $e = send(m)$ ed $e' = receive(m)$, allora $e \rightarrow e'$;
3. se $e \rightarrow e'$ ed $e' \rightarrow e''$, allora $e \rightarrow e''$ (transitività).

Due eventi sono **concorrenti**, e si scrive $e \parallel e'$, se non vale né $e \to e'$ né $e' \to e$.

$e \to e'$ si legge «e precede causalmente e'»: l'esecuzione di e, e il suo risultato, *possono* aver influenzato e'. Solo per una coppia invio–ricezione l'influenza è certa; in generale si può dire soltanto che è possibile. Lamport, che ha introdotto la relazione, la chiamava *happened before*; qui si evita quel nome perché suggerisce un ordine nel tempo reale, mentre conta solo la possibilità di un'influenza. Due eventi concorrenti, invece, non possono essersi influenzati in nessun verso, e il loro ordine nel tempo reale è irrilevante.

Una computazione distribuita, quindi, non è una sequenza di eventi ma un insieme parzialmente ordinato (*poset*): formalmente è la coppia $(H, \to)$. L'ordine è *parziale* perché le coppie di eventi concorrenti non sono ordinate. L'ordine degli eventi di ciascun processo e gli abbinamenti tra invii e ricezioni sono già contenuti in H, perché gli eventi sono numerati e i messaggi hanno identificatori unici.

1.3.2 Diagrammi spazio-tempo

Una computazione si rappresenta comodamente con un **diagramma spazio-tempo** (*space-time diagram*) come quello della Figura 1.2. Ogni processo è una linea orizzontale, con il tempo che scorre da sinistra a destra; gli eventi sono punti sulla linea, e ogni messaggio è una freccia che parte dall'evento di invio e arriva all'evento di ricezione. Gli eventi interni non hanno frecce.

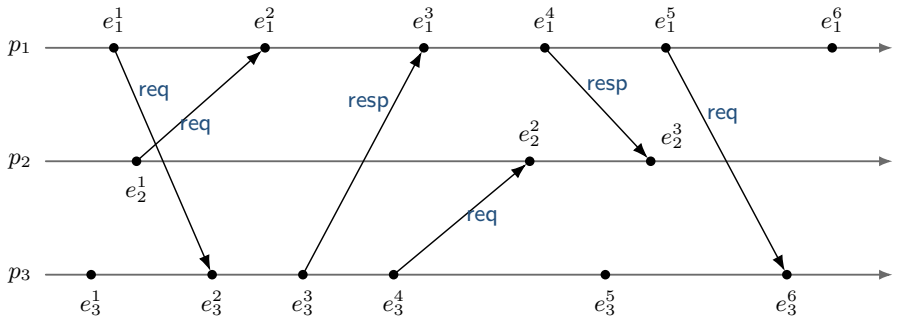


Figura 1.2. La computazione d'esempio: tre processi che si scambiano richieste (req) e risposte (resp).

Esempio (la computazione d'esempio). La computazione della Figura 1.2 tornerà in quasi tutti i capitoli. È un sistema client–server con chiamate di procedura remota: ogni processo può fare sia da client sia da server, e dopo aver inviato una richiesta (req) aspetta la risposta (resp). Per esempio p_1 invia una richiesta a p_3 (e_1^1); p_3 la riceve (e_3^2) e risponde (e_3^3), e la risposta arriva a p_1 in e_1^3. Nel frattempo p_1 ha ricevuto una richiesta da p_2 (e_1^2), a cui risponde in e_1^4. Gli eventi e_3^1, e_3^5 e e_1^6 sono interni.

Il diagramma rende facile leggere la precedenza causale: $e \to e'$ se e solo se si può andare da e a e' con un cammino che si muove verso destra lungo le linee dei

processi e segue le frecce nel loro verso. Se un cammino così non c'è in nessuno dei due versi, i due eventi sono concorrenti.

Esempio (leggere la precedenza causale). Nella Figura 1.2:

- $e_2^1 \to e_3^6$, grazie al cammino $e_2^1 \to e_1^2 \to e_1^3 \to e_1^4 \to e_1^5 \to e_3^6$: la richiesta di p_2 arriva a p_1, si prosegue lungo p_1, e da e_1^5 parte la richiesta che p_3 riceve in e_3^6;
- $e_2^2 \parallel e_3^6$: dopo e_2^2 il processo p_2 non invia più messaggi, quindi da e_2^2 non si arriva a p_3; e e_3^6 è l'ultimo evento di p_3, quindi da lì non si arriva da nessuna parte;
- $e_1^1 \parallel e_2^1$: sono i primi eventi dei rispettivi processi e nessun messaggio li collega.

Capitolo 2

Stati globali, tagli e consistenza

Per valutare un predicato globale bisogna prima ricostruire uno stato globale. Questo capitolo definisce stati locali e globali e i tagli che li individuano, e mostra con un esempio come un monitor ingenuo possa ricostruire uno stato mai esistito e trarne conclusioni sbagliate. La nozione che separa gli stati sensati da quelli impossibili è la consistenza; gli stati consistenti, ordinati, formano il reticolo degli stati globali.

2.1 Stati globali, tagli e run

Indichiamo con σ_i^k lo **stato locale** di p_i subito dopo l'evento e_i^k, e con σ_i^0 il suo stato iniziale, prima di qualunque evento. Lo stato locale comprende i valori delle variabili del processo e, se servono, le sequenze dei messaggi inviati e ricevuti su ciascun canale. Lo **stato globale** è la n-upla degli stati locali, uno per processo: $\Sigma = (\sigma_1, \ldots, \sigma_n)$.

Non serve aggiungere allo stato globale lo stato dei canali, cioè i messaggi in viaggio: se gli stati locali registrano i messaggi inviati e ricevuti, i messaggi in viaggio da p_i a p_j sono quelli che p_i ha inviato e p_j non ha ancora ricevuto. Gli stati dei canali torneranno in modo esplicito con gli snapshot (capitolo 5).

> **Storia e stato.** Storia e stato descrivono la stessa computazione, ma rispondono a domande diverse. La storia dice *che cosa è successo e in che ordine*: è la sequenza degli eventi, come l'elenco delle mosse di una partita a scacchi. Lo stato dice *com'è il processo in un certo punto*: sono i valori delle sue variabili, come la posizione sulla scacchiera.
> Per esempio, p_1 gestisce un conto che parte da 0 euro: riceve da p_2 un bonifico di 50 euro (e_1^1), deposita 100 euro (e_1^2) e ne preleva 30 (e_1^3).
>
> - «Il saldo supera i 100 euro?» Lo stato σ_1^3 lo dice subito: il saldo è 120 euro. Nella storia questo numero non c'è, e per trovarlo bisogna rifare i conti dall'inizio, cioè ricostruire lo stato.
> - «Il bonifico è arrivato prima del prelievo?» La storia lo dice subito. Lo stato no, perché a 120 euro si arriva in molti modi, anche con un solo deposito.

Lo stato σ_i^k corrisponde al prefisso h_i^k dei primi k eventi solo nel senso che ne è il risultato, come dopo k mosse c'è una sola posizione: dalla storia, partendo dallo stato iniziale, si ricava lo stato, ma non il contrario.
Per questo servono tutti e due. I predicati parlano di valori, come i saldi o i processi in attesa, e si valutano sugli stati. L'ordine invece si studia sugli eventi: precedenza causale, clock e consistenza dei tagli (Sezione 2.3) si definiscono su di essi. In GPE la storia dice *dove* si può guardare, cioè quali tagli sono consistenti, e lo stato dice *che cosa* si vede lì, cioè i valori su cui valutare il predicato globale Φ.
Lo stato può contenere anche un pezzo di storia, come le sequenze dei messaggi inviati e ricevuti, perché nessun processo vede la storia: durante l'esecuzione ognuno ha solo il proprio stato e i messaggi che riceve, e ciò che gli serve sapere del passato deve averlo annotato. È così che, come si è appena visto, dagli stati locali si ricavano i messaggi in viaggio, che mancavano nel caso (b) del bonifico (Sezione 1.1).
Annotare il passato, però, non trasforma lo stato nella storia. Se p_i annotasse tutti i propri eventi, σ_i^k conterrebbe h_i^k, ma resterebbe una copia del passato fino a quel punto, fatta da p_i. La storia è invece la descrizione dell'intera computazione vista da fuori: esiste anche se nessuno la annota, ed è il riferimento con cui si giudica se ciò che i processi ricostruiscono è corretto. In pratica, poi, non si annota tutto, perché la storia cresce senza limite, ma solo il riassunto che serve: per il bonifico bastano il totale inviato e il totale ricevuto su ogni canale, e i clock dei capitoli 3 e 4 sono riassunti della storia per ragionare sull'ordine degli eventi (un numero per i clock logici, un vettore con un numero per processo per i *vector clock*).

Per indicare *quale* stato globale si considera basta dire fino a che punto è arrivato ciascun processo. È l'idea di taglio.

Definizione (taglio). Un **taglio** (*cut*) è un sottoinsieme C della storia globale formato da un prefisso della storia locale di ogni processo:

$$C = h_1^{c_1} \cup \cdots \cup h_n^{c_n}.$$

Si indica con la tupla $(c_1, \ldots, c_n)$ degli indici dell'ultimo evento incluso per ciascun processo. Gli ultimi eventi inclusi, $(e_1^{c_1}, \ldots, e_n^{c_n})$, formano la **frontiera** del taglio, e lo stato globale che gli corrisponde è $(\sigma_1^{c_1}, \ldots, \sigma_n^{c_n})$.

Nel diagramma spazio-tempo un taglio è una linea che attraversa dall'alto in basso tutti i processi: gli eventi a sinistra della linea sono nel taglio, quelli a destra no. La Figura 2.1 mostra due tagli della computazione d'esempio, $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

Un taglio dice quali eventi sono avvenuti, non in che ordine. Per ragionare sulle esecuzioni serve anche un ordine totale: in un'esecuzione reale tutti gli eventi, anche di processi diversi, avvengono in un qualche ordine. Due eventi simultanei si possono ordinare in modo arbitrario, per esempio mettendo prima quello del processo con

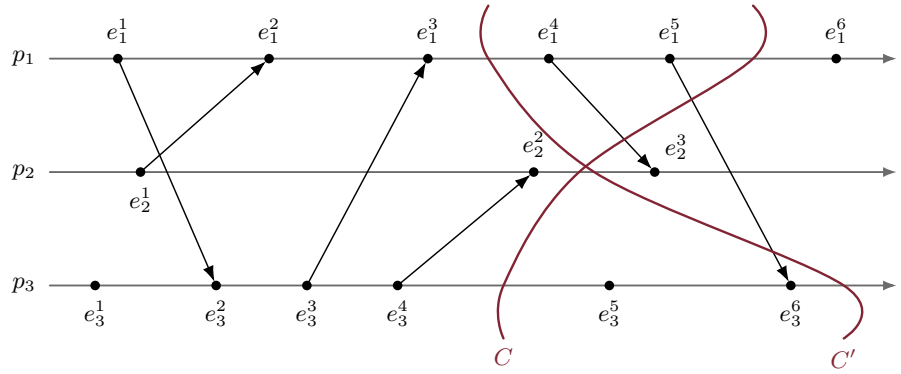


Figura 2.1. Due tagli della computazione d'esempio: $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

indice minore.

Definizione (run). Un **run** (esecuzione) di una computazione è un ordine totale di tutti gli eventi della storia globale che rispetta ogni storia locale: gli eventi di ciascun processo p_i compaiono nel run nello stesso ordine in cui compaiono in h_i.

Una computazione ha in generale molti run, e un run non corrisponde per forza a un'esecuzione possibile: la definizione non impedisce, per esempio, di mettere la ricezione di un messaggio prima del suo invio. A questo rimediano i run consistenti della Sezione 2.3.

2.2 Monitoraggio attivo e deadlock fantasma

Supponiamo che un solo processo, il **monitor** p_0, abbia il compito di valutare il predicato Φ. Può essere uno dei processi $p_1, \ldots, p_n$ oppure un processo esterno alla computazione, ma pur sempre parte del sistema e soggetto ai suoi ritardi. Gli eventi eseguiti per il monitoraggio non fanno parte della computazione e non cambiano la numerazione dei suoi eventi. Con un solo monitor, risolvere GPE significa costruire uno stato globale a cui applicare Φ.

La strategia più semplice è attiva: p_0 chiede lo stato a tutti. Invia a ogni processo una richiesta; quando la riceve, p_i risponde con il proprio stato locale corrente; raccolte tutte le risposte, p_0 compone lo stato globale. I punti in cui le richieste arrivano ai processi definiscono un taglio, e lo stato costruito è quello di quel taglio. Il problema è che le richieste arrivano in momenti scollegati tra loro, quindi il taglio che ne risulta può essere qualunque.

Esempio (deadlock fantasma). Nella computazione d'esempio (Figura 1.2) ogni processo, dopo aver inviato una richiesta, resta bloccato finché non riceve la risposta, quindi il sistema può andare in deadlock. Per rilevarlo, ogni server tiene nel proprio stato locale l'elenco dei client da cui ha ricevuto richieste a cui non ha ancora risposto. Con queste informazioni si costruisce il **grafo delle**

attese (*waits-for graph*, WFG): i nodi sono i processi, e c'è un arco da p_i a p_j se p_j ha ricevuto una richiesta da p_i e non ha ancora risposto, cioè se p_i sta aspettando p_j. Un ciclo nel grafo è un deadlock, perché ogni processo del ciclo aspetta il successivo. Il predicato da valutare è quindi Φ = «il WFG contiene un ciclo».
Supponiamo che le richieste di p_0 arrivino ai processi nei punti del taglio $C' = (3, 2, 6)$ della Figura 2.1, e che p_0 riceva quindi σ_1^3, σ_2^2 e σ_3^6:

- in σ_3^6, p_3 ha ricevuto da p_1 la richiesta inviata in e_1^5 e non ha risposto: arco $p_1 \to p_3$;
- in σ_1^3, p_1 ha ricevuto la richiesta di p_2 (e_1^2) e non ha ancora risposto: arco $p_2 \to p_1$;
- in σ_2^2, p_2 ha ricevuto la richiesta di p_3 (e_2^2) e non ha risposto: arco $p_3 \to p_2$.

Il grafo contiene il ciclo $p_1 \to p_3 \to p_2 \to p_1$ (Figura 2.2) e p_0 dichiara un deadlock. Ma un osservatore esterno che vedesse l'intera computazione direbbe che il sistema non è mai in deadlock: p_1 risponde a p_2 (e_1^4) *prima* di inviare la nuova richiesta a p_3 (e_1^5). Il deadlock rilevato è un **deadlock fantasma** (*ghost deadlock*).

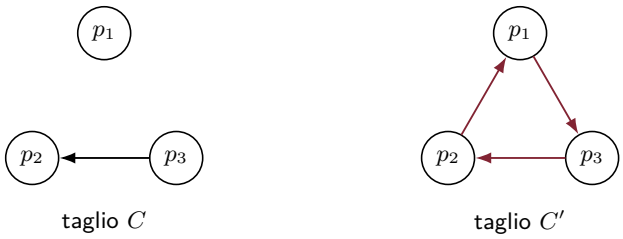


Figura 2.2. Il grafo delle attese costruito sullo stato del taglio C e su quello di C': solo il secondo contiene un ciclo.

Dov'è l'errore? Nello stato di C', p_3 risulta aver ricevuto una richiesta (e_3^6) che p_1 non ha ancora inviato (e_1^5 è fuori dal taglio): uno stato così non si è mai verificato, e non avrebbe mai potuto verificarsi. Se invece le richieste di p_0 arrivassero nei punti del taglio $C = (5, 2, 4)$, il grafo avrebbe il solo arco $p_3 \to p_2$ e nessun ciclo. Ogni taglio corrisponde a uno stato globale, ma solo alcuni corrispondono a stati in cui il sistema si sarebbe potuto trovare davvero. Qui si vedono anche i ruoli di storia e stato: il grafo delle attese si costruisce dagli stati locali, ma l'errore si scopre guardando gli eventi.

2.3 Tagli e run consistenti

La precedenza causale è lo strumento giusto per distinguere i due tipi di taglio.

Definizione (taglio consistente). Un taglio C è **consistente** se, per ogni coppia di eventi e ed e',

$$(e \in C) \wedge (e' \to e) \;\Rightarrow\; e' \in C.$$

In parole: se un evento sta nel taglio, ci stanno anche tutti gli eventi che lo precedono causalmente. Un taglio consistente è «chiuso all'indietro» rispetto alla precedenza causale: non contiene un effetto senza le sue cause.

Dentro un singolo processo questa condizione è sempre soddisfatta, perché un taglio contiene prefissi delle storie locali. L'unico modo per violarla è un messaggio *ricevuto* dentro il taglio ma *inviato* fuori. Nel diagramma il test è immediato: un taglio è consistente se ogni freccia che lo attraversa va da sinistra (dentro il taglio) verso destra (fuori); è inconsistente se una freccia lo attraversa da destra verso sinistra, cioè se c'è un messaggio ricevuto ma non ancora inviato. Nella Figura 2.1 il taglio C è consistente; C' no, perché la richiesta da e_1^5 a e_3^6 lo attraversa all'indietro.

Uno **stato globale consistente** è lo stato di un taglio consistente. Sono esattamente gli stati che un osservatore ideale, esterno al sistema, avrebbe potuto vedere durante un'esecuzione. Il deadlock fantasma nasce dall'aver valutato Φ in uno stato inconsistente: il valore di un predicato ha senso solo negli stati globali consistenti.

I tagli consistenti hanno un ruolo centrale. In un programma sequenziale un istante di tempo individua un punto preciso dell'esecuzione; in una computazione distribuita questo ruolo lo ha la frontiera di un taglio consistente. Anche «prima» e «dopo» vanno intesi rispetto ai tagli consistenti: un evento è prima di un taglio se sta a sinistra della sua frontiera, dopo se sta a destra.

La stessa idea si applica ai run.

Definizione (run consistente). Un run R è **consistente** se, per ogni coppia di eventi, $e \to e'$ implica che e compare prima di e' in R.

In altre parole, l'ordine totale del run estende l'ordine parziale della precedenza causale. Un run $R = e^1 e^2 \dots$ si può vedere anche come una sequenza di stati globali $\Sigma^0 \Sigma^1 \Sigma^2 \dots$, dove $\Sigma^0 = (\sigma_1^0, \dots, \sigma_n^0)$ è lo stato iniziale e ogni Σ^i si ottiene da Σ^{i-1} eseguendo il singolo evento e^i. Se il run è consistente, ogni suo prefisso è un taglio consistente, quindi tutti gli stati della sequenza sono consistenti.

Due stati consecutivi di un run sono legati dalla relazione *leads to* («porta a»): Σ^{i-1} porta a Σ^i nel run R. La sua chiusura transitiva si scrive $\rightsquigarrow_R$, e Σ' è **raggiungibile** da Σ nel run R se $\Sigma \rightsquigarrow_R \Sigma'$. Si scrive semplicemente $\Sigma \rightsquigarrow \Sigma'$ se esiste *qualche* run in cui Σ' è raggiungibile da Σ.

2.4 Il reticolo degli stati globali

L'insieme degli stati globali consistenti di una computazione, con la relazione *leads to*, forma un **reticolo** (*lattice*). È un modo di disegnare tutti gli stati in cui il sistema si può trovare e tutti i modi di passare dall'uno all'altro.

La notazione. Si scrive $\Sigma^{k_1 \cdots k_n}$ per lo stato globale $(\sigma_1^{k_1}, \ldots, \sigma_n^{k_n})$, quello in cui p_1 ha eseguito esattamente k_1 eventi, p_2 esattamente k_2, e così via: è lo stato del taglio $(k_1, \ldots, k_n)$. Con due processi si scrive Σ^{ab}, e le due cifre in apice sono due contatori separati, non un numero: Σ^{21} si legge «p_1 ha eseguito due eventi, p_2 uno», e Σ^{00} è lo stato iniziale. Il **livello** di uno stato è la somma $k_1 + \cdots + k_n$, cioè il numero totale di eventi eseguiti: Σ^{21} sta al livello 3.

> **Due notazioni da non confondere.** Nella Sezione 2.3 gli stati attraversati da un run sono scritti $\Sigma^0\Sigma^1\Sigma^2\ldots$: lì l'unico indice conta gli eventi eseguiti *lungo quel run*, di qualunque processo siano. In $\Sigma^{k_1 \cdots k_n}$, invece, c'è un indice per ogni processo. Per esempio, se un run comincia con $e_2^1\, e_1^1\, e_1^2$, il suo Σ^3 è Σ^{21}.

Come si costruisce. Prendiamo la computazione di due processi della Figura 2.3, a destra: p_1 ha sei eventi, p_2 cinque, e ci sono tre messaggi. Il reticolo si costruisce dall'alto.

1. In cima si mette lo stato iniziale Σ^{00}.
2. Da ogni stato si scende di un passo eseguendo un solo evento in più: il prossimo di p_1, e allora nella figura si scende a sinistra, da Σ^{ab} a $\Sigma^{a+1,b}$; oppure il prossimo di p_2, e si scende a destra, verso $\Sigma^{a,b+1}$.
3. Un passo è possibile solo se lo stato che si ottiene è consistente: un evento interno o un invio si può eseguire sempre, una ricezione solo se l'invio corrispondente è già stato eseguito.
4. Gli stati dello stesso livello stanno sulla stessa riga.

Con n processi il reticolo ha n assi, uno per processo, e già con tre processi è difficile da disegnare, ma la costruzione è la stessa.

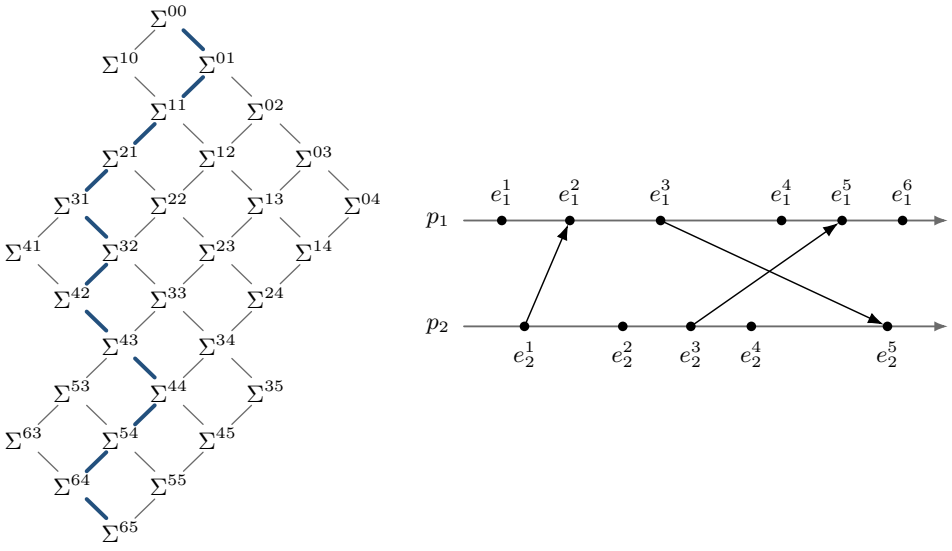


Figura 2.3. Una computazione di due processi (a destra) e il reticolo dei suoi stati globali consistenti (a sinistra). In blu un cammino, cioè un run consistente.

La Figura 2.4 mostra i primi livelli, con l'evento eseguito a ogni passo. Da Σ^{00} si può eseguire e_1^1 oppure e_2^1. Da Σ^{10}, invece, non si può scendere a sinistra: il prossimo evento di p_1 è e_1^2, che riceve il messaggio inviato in e_2^1, non ancora eseguito. Lo stato Σ^{20}, quindi, non c'è. Da Σ^{11} lo stesso evento e_1^2 è possibile, perché ora e_2^1 è avvenuto, e si arriva a Σ^{21}.

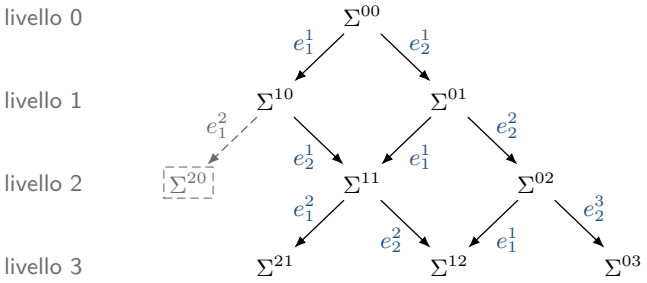


Figura 2.4. I primi livelli del reticolo della Figura 2.3, con l'evento eseguito a ogni passo. Da Σ^{10} non si può eseguire e_1^2, che riceve un messaggio non ancora inviato: Σ^{20} non è consistente.

Quali stati mancano lo dicono i messaggi, perché ognuno impone che la sua ricezione non avvenga prima dell'invio. Nella computazione della figura i vincoli su Σ^{ab} sono tre:

- per il messaggio da e_2^1 a e_1^2: se $a \geq 2$, allora $b \geq 1$;
- per il messaggio da e_2^3 a e_1^5: se $a \geq 5$, allora $b \geq 3$;
- per il messaggio da e_1^3 a e_2^5: se $b \geq 5$, allora $a \geq 3$.

Nel reticolo completo della Figura 2.3 mancano quindi gli stati con $a \geq 2$ e $b = 0$, come Σ^{20}; quelli con $a \geq 5$ e $b \leq 2$, come Σ^{51}; e quelli con $b = 5$ e $a \leq 2$, come Σ^{05}. Tutti gli altri ci sono, e ognuno è raggiungibile dallo stato iniziale Σ^{00}.

Cammini e run. Un **cammino** nel reticolo è una sequenza di stati in cui ogni stato ha un evento in più del precedente: a ogni passo si sceglie quale processo avanza. Ogni cammino dallo stato iniziale a quello finale corrisponde a un run consistente, e ogni run consistente è un cammino. Il cammino evidenziato in blu nella Figura 2.3, per esempio, passa per gli stati

$$\Sigma^{00}\,\Sigma^{01}\,\Sigma^{11}\,\Sigma^{21}\,\Sigma^{31}\,\Sigma^{32}\,\Sigma^{42}\,\Sigma^{43}\,\Sigma^{44}\,\Sigma^{54}\,\Sigma^{64}\,\Sigma^{65}$$

e corrisponde al run

$$e_2^1\,e_1^1\,e_1^2\,e_1^3\,e_2^2\,e_1^4\,e_2^3\,e_2^4\,e_1^5\,e_1^6\,e_2^5.$$

Ogni passo a sinistra è un evento di p_1, ogni passo a destra un evento di p_2. Cammini diversi sono ordini diversi in cui si possono eseguire gli eventi concorrenti, e descrivono tutti la stessa computazione. Si sarebbe tentati di cercare quale di questi cammini sia l'esecuzione *effettiva*. In un sistema asincrono, però, dall'interno del sistema è impossibile saperlo: solo un osservatore esterno onnisciente conoscerebbe gli stati per cui l'esecuzione è passata davvero. Per chi sta dentro il sistema, tutti i cammini sono ugualmente plausibili.

Raggiungibilità. Nel reticolo, $\Sigma \rightsquigarrow \Sigma'$ vuol dire che da Σ si può scendere fino a Σ' seguendo gli archi. Succede esattamente quando in Σ' ogni processo ha eseguito almeno tanti eventi quanti in Σ. Per esempio $\Sigma^{21} \rightsquigarrow \Sigma^{23}$, passando per Σ^{22}; invece tra Σ^{31} e Σ^{23} nessuno dei due è raggiungibile dall'altro, perché il primo ha più eventi di p_1 e il secondo più eventi di p_2.

Tre proprietà dei tagli consistenti aiutano a ragionare sul reticolo.

- *Intersezione e unione di due tagli consistenti sono tagli consistenti.* Se un evento sta in entrambi i tagli, le sue cause stanno in entrambi; se sta in almeno uno dei due, le sue cause stanno almeno in quello. È questo a fare dell'insieme un reticolo: per due stati consistenti esistono sempre il più grande stato da cui entrambi sono raggiungibili (quello dell'intersezione, il minimo componente per componente) e il più piccolo stato raggiungibile da entrambi (quello dell'unione, il massimo). Per Σ^{31} e Σ^{23} sono Σ^{21} e Σ^{33}.
- *Ogni stato globale consistente è raggiungibile dallo stato iniziale con un run consistente.* Basta eseguire gli eventi del taglio uno alla volta, scegliendo ogni volta un evento le cui cause sono già state eseguite. Un evento così esiste sempre, perché la precedenza causale non ha cicli.
- *Se $C \subseteq C'$ sono due tagli consistenti, lo stato di C' è raggiungibile da quello di C.* Si prende un run consistente che arriva allo stato di C e lo si prolunga con gli eventi di $C' \setminus C$, sempre in un ordine che rispetta la precedenza causale.

L'ultima proprietà tornerà utile con gli snapshot (Sezione 5.6).

Capitolo 3

Osservare una computazione: orologio reale e clock logici

Nel capitolo precedente il monitor interrogava attivamente i processi e otteneva un taglio qualunque. Qui si prova la strategia opposta: il monitor non chiede nulla, sono i processi a informarlo di ogni evento, e il suo compito diventa mettere in ordine le notifiche in modo che la sequenza osservata sia consistente. Si parte da ipotesi forti, un orologio globale e ritardi limitati, e le si toglie una alla volta. Lo strumento centrale del capitolo sono i clock logici di Lamport.

3.1 Monitoraggio passivo e osservazioni

Nella strategia passiva il monitor p_0 non invia messaggi. I processi della computazione vengono modificati di poco: ogni volta che eseguono un evento, lo notificano a p_0 con un messaggio che lo descrive. L'invio della notifica non è un evento nuovo, ma coincide con l'evento notificato. In pratica basta notificare gli eventi **rilevanti** per Φ, cioè quelli che possono cambiarne il valore: per il deadlock della computazione d'esempio sono gli invii e le ricezioni di richieste e risposte, gli unici che modificano il grafo delle attese. Il monitor costruisce così un'**osservazione** della computazione: la sequenza degli eventi nell'ordine in cui arrivano le notifiche.

Le osservazioni hanno due caratteristiche importanti.

- Poiché i ritardi variano, la stessa esecuzione può produrre osservazioni diverse su monitor diversi. È il cosiddetto **effetto relativistico** del calcolo distribuito: non esiste un punto di vista privilegiato.
- Un'osservazione può corrispondere a un run consistente, a un run inconsistente o a nessun run, se le notifiche di uno stesso processo arrivano in disordine. Si dice **consistente** un'osservazione che corrisponde a un run consistente.

Esempio (tre osservazioni della stessa esecuzione). Supponiamo che la computazione d'esempio (Figura 1.2) venga notificata a p_0. Con canali non FIFO qualunque ordine di arrivo delle notifiche è possibile.

- Se la notifica di e_3^4 arriva prima di quella di e_3^3, la sequenza osservata non

rispetta nemmeno la storia locale di p_3: non è un run.

- Se le notifiche di e_1^4 ed e_1^5 tardano mentre quella di e_3^6 arriva presto, p_0 vede p_3 ricevere una richiesta che p_1 non ha ancora inviato: l'osservazione è un run inconsistente. Se a quel punto p_0 valuta Φ, si trova proprio nello stato del taglio C', cioè $(\sigma_1^3, \sigma_2^2, \sigma_3^6)$, e ritrova il deadlock fantasma.
- Se le notifiche arrivano in un ordine che rispetta la precedenza causale, l'osservazione è consistente, e tutti gli stati che p_0 attraversa sono consistenti: per esempio quello del taglio C.

3.2 Regole di consegna e consegna FIFO

Per rimettere in ordine i messaggi, chi li riceve non li passa subito all'applicazione, ma decide quando farlo seguendo una **regola di consegna** (*delivery rule*). Si distinguono quindi due operazioni: con *receive* il messaggio arriva dal canale, con *deliver* viene consegnato all'applicazione. La ricezione resta nascosta dentro la regola di consegna e non compare nella storia locale del processo: per l'applicazione conta solo la consegna.

Definizione (consegna FIFO). La comunicazione da p_i a p_j rispetta la **consegna FIFO** (*First In, First Out*) se, per ogni coppia di messaggi m ed m',

$$send_i(m) \to send_i(m') \;\Rightarrow\; deliver_j(m) \to deliver_j(m'),$$

dove il pedice indica il processo che esegue l'evento.

In parole: un messaggio non può sorpassare un messaggio inviato prima dallo stesso mittente. La consegna FIFO si realizza facilmente anche su canali che non la garantiscono: il mittente numera i messaggi che invia a ciascun destinatario, e il destinatario li consegna in ordine di numero, tenendo da parte quelli arrivati in anticipo. Se per esempio arrivano prima i messaggi numero 2 e 3, il destinatario li trattiene finché non arriva l'1, e poi consegna 1, 2 e 3.

Con canali FIFO tra i processi e il monitor, le notifiche di ogni processo arrivano nell'ordine giusto e ogni osservazione è un run: il primo caso dell'esempio non può più capitare. La consistenza però non è garantita: il secondo caso riguarda notifiche di processi *diversi*, p_1 e p_3, che la consegna FIFO non ordina tra loro. Serve un meccanismo che ordini le notifiche secondo la precedenza causale.

3.3 Orologio reale e clock condition

Partiamo da ipotesi forti, che poi verranno tolte. Supponiamo che tutti i processi possano leggere un orologio reale globale e che ogni messaggio arrivi entro un ritardo massimo noto δ. Non è più un sistema asincrono, ma è un buon punto di partenza. Sia $RC(e)$ il valore dell'orologio quando viene eseguito l'evento e: ogni notifica porta con sé come **timestamp** (marca temporale) il valore $RC(e)$ dell'evento notificato, e il monitor segue questa regola.

Definizione (regola DR1). Al tempo t, consegna tutti i messaggi ricevuti con timestamp fino a $t - \delta$, in ordine di timestamp crescente.

Perché funziona? Un messaggio con timestamp non superiore a $t - \delta$ è stato inviato almeno δ prima di t, quindi al tempo t è sicuramente già arrivato. Quando il monitor consegna i messaggi con timestamp fino a $t - \delta$, perciò, non può più arrivare un messaggio con timestamp minore di quelli già consegnati. L'osservazione ordina quindi gli eventi per timestamp crescente: e viene osservato prima di e' se e solo se $RC(e) < RC(e')$ (a parità di timestamp decide l'indice del processo). Allora l'osservazione è consistente se e solo se i timestamp rispettano la precedenza causale.

Definizione (clock condition). Una funzione T che associa un valore a ogni evento soddisfa la **clock condition** se

$$e \to e' \;\Rightarrow\; T(e) < T(e').$$

L'orologio reale la soddisfa senza fatica, perché una causa avviene sempre prima del suo effetto. L'osservazione chiave è che la clock condition è l'*unica* proprietà dell'orologio reale che serve: qualunque meccanismo di timestamp che la soddisfi si può usare al posto del tempo reale. E un meccanismo così esiste anche in un sistema asincrono.

Attenzione al verso dell'implicazione: la clock condition dice che le cause hanno timestamp più piccoli degli effetti, non che un timestamp più piccolo indichi una causa. Due eventi con $T(e) < T(e')$ possono essere legati causalmente oppure concorrenti; l'unica cosa certa è che e' non precede causalmente e.

3.4 Clock logici di Lamport

Il clock logico di Lamport è un contatore che ogni processo mantiene per conto proprio e che fa da orologio senza alcuna sincronizzazione tra i processi.

Definizione (clock logico di Lamport). Ogni processo p_i mantiene un contatore LC, il suo **clock logico** (*logical clock*), inizialmente 0. Ogni messaggio m porta un timestamp $TS(m)$, il valore del clock del mittente nell'evento di invio. A ogni nuovo evento e_i, p_i aggiorna il clock così:

$$LC(e_i) = \begin{cases} LC + 1 & \text{se } e_i \text{ è un evento interno o un invio,} \\ \max\{LC, TS(m)\} + 1 & \text{se } e_i = \mathit{receive}(m). \end{cases}$$

Un evento interno o un invio fa avanzare il clock di uno. Una ricezione porta il clock oltre sia il proprio valore precedente sia il timestamp del messaggio: il destinatario salta in avanti per non restare indietro rispetto al passato del mittente.

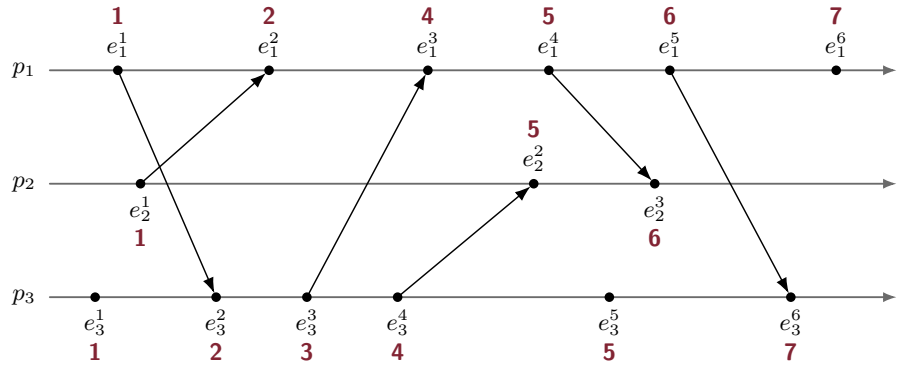


Figura 3.1. I clock logici della computazione d'esempio: accanto a ogni evento, in rosso, il valore di LC.

Esempio (clock logici passo passo). Applichiamo le regole alla computazione d'esempio (Figura 3.1). All'inizio tutti i clock valgono 0.

- I primi eventi e_1^1, e_2^1 ed e_3^1 sono invii o eventi interni, e il loro clock vale 1. Le richieste inviate in e_1^1 ed e_2^1 portano timestamp 1.
- e_1^2 riceve la richiesta di p_2, con timestamp 1: $\max\{1,1\}+1=2$. Allo stesso modo e_3^2, che riceve la richiesta di p_1, vale 2.
- p_3 invia la risposta, $LC(e_3^3)=3$, e poi una richiesta a p_2, $LC(e_3^4)=4$.
- e_1^3 riceve la risposta di p_3, con timestamp 3: $\max\{2,3\}+1=4$. Il clock di p_1 salta da 2 a 4 per superare il passato di p_3.
- e_2^2 riceve la richiesta di p_3, con timestamp 4: $\max\{1,4\}+1=5$.
- p_1 prosegue con la risposta a p_2, $LC(e_1^4)=5$, la richiesta a p_3, $LC(e_1^5)=6$, e l'evento interno e_1^6, che vale 7.
- e_2^3 riceve la risposta di p_1, con timestamp 5: $\max\{5,5\}+1=6$.
- e_3^5 è interno e vale 5; e_3^6 riceve la richiesta inviata in e_1^5, con timestamp 6: $\max\{5,6\}+1=7$. Il clock di p_3 passa da 5 a 7 e non assume mai il valore 6.

I clock logici crescono lungo ogni catena causale: a ogni passo dentro un processo il clock aumenta di uno, e a ogni messaggio il clock del destinatario supera il timestamp del mittente. Ne segue la proprietà fondamentale.

I clock logici soddisfano la clock condition. Per ogni coppia di eventi

$$e \to e' \;\Rightarrow\; LC(e) < LC(e').$$

Il viceversa non vale, e dai soli valori dei clock non si capisce se due eventi siano legati o concorrenti. Nella Figura 3.1, $LC(e_2^2)=5<LC(e_3^6)=7$, ma i due eventi sono concorrenti, come si è visto nel capitolo 1. Da $LC(e)<LC(e')$ si deduce

soltanto che e' non precede causalmente e.

Ordinando tutti gli eventi per clock logico crescente, e risolvendo le parità con l'indice del processo, si ottiene un ordine totale che rispetta la precedenza causale, cioè un run consistente. Per la computazione d'esempio è

$$e_1^1\, e_2^1\, e_3^1\, e_1^2\, e_3^2\, e_3^3\, e_1^3\, e_3^4\, e_1^4\, e_2^2\, e_3^5\, e_1^5\, e_2^3\, e_1^6\, e_3^6.$$

3.5 Consegnare le notifiche in ordine di clock logico

Torniamo al monitor. Se i processi usano i clock logici come timestamp delle notifiche e p_0 le consegna in ordine di timestamp crescente, l'osservazione è consistente, perché i clock logici soddisfano la clock condition. Resta però un problema serio: senza un limite ai ritardi, p_0 non può mai essere sicuro che più tardi non arrivi una notifica con timestamp minore di quelle che ha in mano. Per prudenza non consegnerebbe mai nulla: la regola non ha *liveness*, cioè non garantisce che prima o poi qualcosa venga consegnato. Manca la capacità di accorgersi dei «buchi».

Definizione (gap detection). Un meccanismo di clock ha la proprietà di **gap detection** se, dati due eventi e ed e' con $LC(e) < LC(e')$, permette di stabilire se esiste un altro evento e'' con $LC(e) < LC(e'') < LC(e')$.

I clock logici non hanno questa proprietà: i valori possono saltare, come il clock di p_1 che passa da 2 a 4, e guardando i soli timestamp non si sa se da qualche parte ci sia un evento con valore 3 di cui non è ancora arrivata la notifica. Nemmeno l'orologio reale ha la gap detection, ma con il limite δ sui ritardi il controllo diventa facile: al tempo t sono arrivati tutti i messaggi con timestamp minore di $t - \delta$.

Con i clock logici serve un'informazione in più, e la fornisce la consegna FIFO. Supponiamo che i canali da ogni processo a p_0 siano FIFO. Poiché il clock di ogni processo cresce e i messaggi di uno stesso mittente arrivano in ordine, quando p_0 riceve da p_i un messaggio con un certo timestamp sa che da p_i non arriveranno più messaggi con timestamp minore o uguale. Un messaggio m ricevuto da p_0 si dice **stabile** se non può più arrivare un messaggio con timestamp minore di $TS(m)$. Con canali FIFO, m è stabile non appena p_0 ha ricevuto da *ogni* altro processo almeno un messaggio con timestamp maggiore di $TS(m)$.

Definizione (regola DR2). Consegna tutti i messaggi ricevuti che sono stabili per p_0, in ordine di timestamp crescente.

La regola si può ancora bloccare se un processo smette di inviare notifiche, perché p_0 non riceverebbe mai da lui un timestamp più grande. Si rimedia facendo chiedere a p_0, periodicamente, una conferma a tutti i processi: le risposte hanno timestamp più grandi e rendono stabili i messaggi precedenti.

DR2 funziona, ma è prudente: consegna in ordine di timestamp anche le notifiche di eventi concorrenti, che si potrebbero consegnare in qualunque ordine. Il capitolo 4 mostra come fare di meglio con un clock più informativo.

Capitolo 4

Consegna causale e vector clock

Nel capitolo precedente il monitor p_0 riceveva dai processi una notifica per ogni evento, e doveva metterle in un ordine che rispettasse la precedenza causale, così da osservare un run consistente. Con i clock logici ci riesce, ma a un prezzo: consegna le notifiche in ordine di timestamp e aspetta anche quando non serve, perché un clock logico non dice se due eventi sono legati causalmente o concorrenti. Questo capitolo introduce la consegna causale, che è esattamente la proprietà di cui ha bisogno il monitor, e i *vector clock* (clock vettoriali): timestamp da cui si capisce sempre se due eventi sono legati o concorrenti, e con cui la consegna causale si realizza senza attese inutili.

4.1 Consegna causale

Ricordiamo la distinzione della Sezione 3.2: un messaggio viene *ricevuto* quando arriva dal canale, e viene *consegnato* quando il processo lo passa all'applicazione; tra le due cose il processo può tenerlo da parte. La consegna FIFO garantisce che i messaggi di uno stesso mittente siano consegnati nell'ordine in cui sono stati inviati. Quando i messaggi vengono da mittenti diversi, però, non basta.

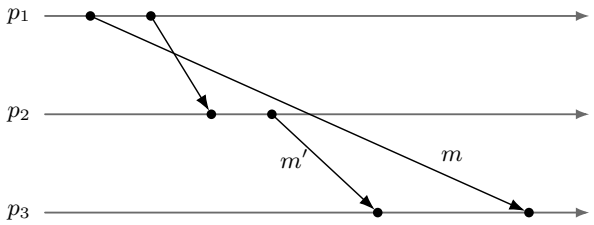


Figura 4.1. Consegna FIFO ma non causale: l'invio di m precede causalmente quello di m', ma p_3 riceve m' per primo.

Nella Figura 4.1, p_1 invia m a p_3 e poi un messaggio a p_2; p_2, dopo averlo ricevuto, invia m' a p_3. Il messaggio m' è quindi una conseguenza di m: p_2 lo invia dopo aver ricevuto qualcosa che p_1 ha mandato *dopo* m. In termini di precedenza causale, l'invio di m precede causalmente l'invio di m'. Eppure m' arriva a p_3 per primo. Su ogni canale viaggia un solo messaggio, quindi la consegna FIFO è rispettata comunque; ma se p_3 consegna subito m', vede l'effetto prima della causa. La consegna

causale chiede proprio che questo non succeda: se l'invio di un messaggio precede causalmente l'invio di un altro, e i due messaggi vanno allo stesso destinatario, il primo deve essere consegnato prima del secondo.

Definizione (consegna causale). Una regola di consegna rispetta la **consegna causale** (*causal delivery*, CD) se, per ogni coppia di messaggi m ed m' diretti allo stesso processo p_k,

$$send_i(m) \to send_j(m') \;\Rightarrow\; deliver_k(m) \to deliver_k(m').$$

Qui $send_i(m)$ è l'evento in cui p_i invia m, $send_j(m')$ quello in cui p_j invia m', e $deliver_k(m)$ quello in cui p_k consegna m: il pedice indica sempre il processo che esegue l'evento.

Se m ed m' hanno lo stesso mittente, cioè $i = j$, si ritrova la consegna FIFO, che quindi è un caso particolare della consegna causale.

Per il monitor la consegna causale è esattamente ciò che serve. La notifica di un evento parte proprio in quell'evento, quindi se $e \to e'$ anche l'invio della notifica di e precede causalmente quello della notifica di e'. Se p_0 consegna le notifiche rispettando la consegna causale, vede sempre le cause prima degli effetti, e tutte le sue osservazioni sono consistenti. Le regole DR1 e DR2 del capitolo precedente sono modi di realizzare la consegna causale sotto ipotesi particolari: un orologio globale con ritardi limitati, oppure canali FIFO e clock logici. Ora la si vuole ottenere con le sole ipotesi di un sistema asincrono, e senza attese inutili.

4.2 La strong clock condition

Perché DR1 e DR2 aspettano anche quando non serve? Consegnano le notifiche in ordine di timestamp crescente, e quindi trattano ogni notifica con un timestamp più piccolo come se potesse essere una causa. Vediamo un caso concreto con i clock logici della Figura 3.1: $LC(e_2^2) = 5$ e $LC(e_3^6) = 7$. Supponiamo che a p_0 arrivi la notifica di e_3^6 prima di quella di e_2^2. Con le regole basate sui clock logici, p_0 non può consegnare e_3^6 finché non ha consegnato e_2^2, che ha un timestamp più piccolo e quindi potrebbe essere una sua causa. In realtà non lo è: i due eventi sono concorrenti, come si è visto nel capitolo 1, e consegnare subito e_3^6 sarebbe stato corretto. Il problema è che il valore 5, da solo, non dice se e_2^2 è una causa di e_3^6 oppure un evento concorrente. Servirebbe un timestamp più informativo, da cui si capisca *esattamente* quando due eventi sono legati causalmente.

Definizione (strong clock condition). Una funzione T che associa un timestamp a ogni evento soddisfa la **strong clock condition** se, per ogni coppia di eventi,

$$e \to e' \iff T(e) < T(e').$$

La doppia freccia $\iff$ si legge «se e solo se»: l'implicazione vale nei due versi. Il verso da sinistra a destra è la clock condition del capitolo 3; quello nuovo dice che, se il timestamp di e è più piccolo di quello di e', allora e è davvero una causa di e'.

Così il confronto tra due timestamp dice sempre come stanno due eventi: se uno dei due timestamp è più piccolo, quell'evento precede l'altro; se nessuno dei due è più piccolo, gli eventi sono concorrenti. Si dice che un timestamp così **caratterizza** la precedenza causale: dai soli timestamp si ricostruisce l'intero ordine parziale $(H, \to)$.

Un solo numero non basta: due numeri diversi sono sempre confrontabili, mentre i timestamp di due eventi concorrenti non devono esserlo. Servono timestamp con più dimensioni, che possano risultare incomparabili: è l'idea dei vector clock, a cui si arriva passando per le storie causali.

4.3 Storie causali

Il modo più diretto per soddisfare la strong clock condition è usare come timestamp di un evento il suo intero passato, cioè l'insieme di tutti gli eventi che possono averlo influenzato.

Definizione (storia causale). La **storia causale** (*causal history*) di un evento e è l'insieme formato da e e da tutti gli eventi che lo precedono causalmente:

$$\theta(e) = \{\, e' \in H \mid e' \to e \,\} \cup \{e\}.$$

La formula si legge così: $\theta(e)$ (con la lettera greca *theta*) è l'insieme degli eventi e' della storia globale H tali che e' precede causalmente e, a cui si aggiunge e stesso.

Nella Figura 4.2 è evidenziata la storia causale di e_1^4:

$$\theta(e_1^4) = \{e_1^1, e_1^2, e_1^3, e_1^4, e_2^1, e_3^1, e_3^2, e_3^3\}.$$

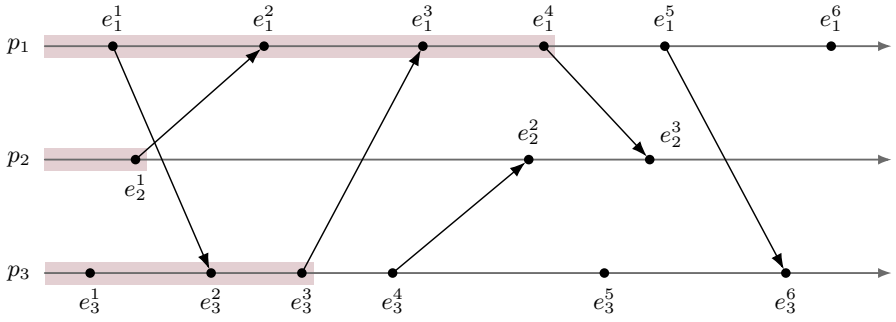


Figura 4.2. La storia causale di e_1^4: i tratti evidenziati delle storie locali.

Usate come timestamp, le storie causali si confrontano per contenimento:

$$e \to e' \iff \theta(e) \subset \theta(e'),$$

dove $\subset$ vuol dire «contenuto e diverso»: e precede e' se e solo se tutto il passato di e fa parte del passato di e'. Se nessuna delle due storie è contenuta nell'altra, gli eventi sono concorrenti. La strong clock condition, quindi, è soddisfatta. Il difetto è pratico: le storie causali crescono senza limite, e ogni messaggio dovrebbe trasportare insiemi sempre più grandi.

4.4 Vector clock

Una storia causale si può riassumere con pochi numeri. Guardiamo che cosa contiene su un singolo processo. Di p_3, la storia causale di e_1^4 contiene e_3^1, e_3^2 ed e_3^3: i primi tre eventi, senza buchi. Non è un caso. Se la storia causale contiene un evento di p_3, contiene anche tutti gli eventi precedenti di p_3, perché anche loro lo precedono causalmente. Su ogni processo, quindi, la storia causale contiene sempre i primi eventi della storia locale, e per descriverla basta dire quanti sono. Per e_1^4 sono 4 eventi di p_1, 1 di p_2 e 3 di p_3: la sua storia causale si riassume nel vettore $(4, 1, 3)$.

Definizione (vector clock). Il **vector clock** di un evento e è il vettore $VC(e)$ di n numeri naturali, uno per processo: la componente i, indicata con $VC(e)[i]$, è il numero di eventi di p_i che stanno nella storia causale di e.
Si calcola mentre la computazione procede. Ogni processo p_i tiene un vettore VC, inizialmente tutto a zero, e ogni messaggio m porta come timestamp $TS(m)$ il vector clock dell'evento in cui è stato inviato. A ogni nuovo evento, p_i aggiorna il vettore così:

- se l'evento è interno o un invio, incrementa la propria componente: $VC[i] := VC[i] + 1$;
- se l'evento è la ricezione di un messaggio m, prima prende il massimo componente per componente tra il proprio vettore e il timestamp del messaggio, $VC := \max\{VC, TS(m)\}$, e poi incrementa la propria componente, $VC[i] := VC[i] + 1$.

Nelle regole, VC senza argomento è il vettore corrente del processo: dopo un evento e il suo valore è proprio $VC(e)$. Il simbolo $:=$ indica un assegnamento, e il massimo componente per componente si calcola posizione per posizione: per esempio $\max\{(2, 1, 0), (1, 0, 3)\} = (2, 1, 3)$. L'idea è che una ricezione unisce ciò che sapeva il destinatario con ciò che sapeva il mittente quando ha inviato il messaggio. Attenzione: alla ricezione si incrementa solo la componente del processo che riceve, non tutte.

Per un evento e di p_i, quindi, la componente del processo stesso, $VC(e)[i]$, è la posizione di e nella storia di p_i; ogni altra componente $VC(e)[j]$ dice quanti eventi di p_j precedono causalmente e, cioè quanto p_i «sa» di p_j al momento di e.

Esempio (vector clock passo passo). Applichiamo le regole alla computazione d'esempio (Figura 4.3); i vettori hanno tre componenti, una per processo.

- e_1^1, e_2^1 ed e_3^1 sono i primi eventi dei rispettivi processi, con vector clock $(1, 0, 0)$, $(0, 1, 0)$ e $(0, 0, 1)$.
- e_1^2 riceve la richiesta di p_2, con timestamp $(0, 1, 0)$: il massimo con $(1, 0, 0)$ è $(1, 1, 0)$, e incrementando la componente di p_1 si ottiene $(2, 1, 0)$.
- e_3^2 riceve la richiesta di p_1, con timestamp $(1, 0, 0)$: dal massimo con $(0, 0, 1)$, cioè $(1, 0, 1)$, si ottiene $(1, 0, 2)$. Seguono due invii, con vector clock $(1, 0, 3)$ e $(1, 0, 4)$.

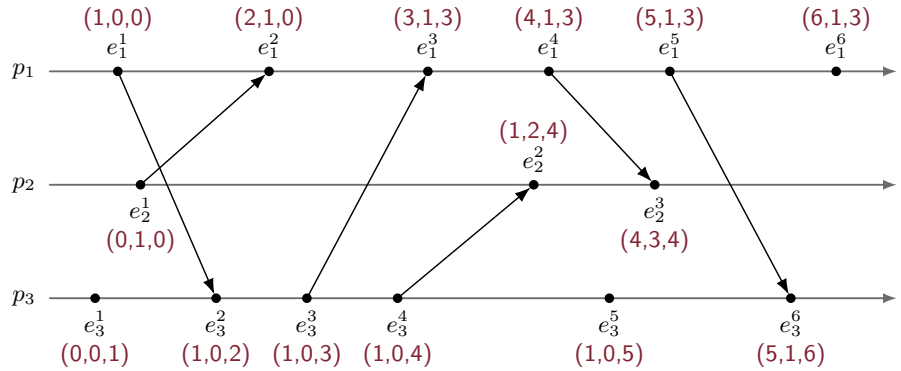


Figura 4.3. I vector clock della computazione d'esempio: accanto a ogni evento, in rosso, il suo vector clock.

- e_1^3 riceve la risposta di p_3, con timestamp $(1,0,3)$: dal massimo con $(2,1,0)$, cioè $(2,1,3)$, si ottiene $(3,1,3)$. Gli eventi successivi di p_1 non ricevono nulla e incrementano solo la prima componente: $(4,1,3)$, $(5,1,3)$, $(6,1,3)$.
- e_2^2 riceve la richiesta di p_3, con timestamp $(1,0,4)$, e vale $(1,2,4)$; e_2^3 riceve la risposta di p_1, con timestamp $(4,1,3)$: il massimo con $(1,2,4)$ è $(4,2,4)$, quindi $(4,3,4)$.
- e_3^5 è interno, $(1,0,5)$; e_3^6 riceve la richiesta di p_1, con timestamp $(5,1,3)$, e vale $(5,1,6)$.

Per esempio $VC(e_2^3) = (4,3,4)$ dice che e_2^3 è il terzo evento di p_2 e che lo precedono causalmente quattro eventi di p_1 (e_1^1–e_1^4) e quattro di p_3 (e_3^1–e_3^4).

4.5 Confrontare i vector clock

Due vettori V e V' di n numeri si confrontano una posizione alla volta:

- $V \leq V'$ se ogni componente di V è minore o uguale alla corrispondente di V';
- $V < V'$ se $V \leq V'$ e i due vettori sono diversi, cioè almeno una componente di V è strettamente minore.

Per esempio $(1,0,0) < (2,1,0)$. A differenza dei numeri, due vettori possono essere **incomparabili**: $(2,1,0)$ e $(1,0,4)$ non sono uno minore dell'altro, perché il primo è più grande nella prima componente e il secondo nella terza. È proprio ciò che mancava per rappresentare la concorrenza.

Vediamo che cosa si capisce dai vector clock, con quattro domande. Gli esempi usano i valori della Figura 4.3.

Un evento precede l'altro? Sì, se e solo se il suo vector clock è più piccolo: i vector clock soddisfano la strong clock condition,

$$e \to e' \iff VC(e) < VC(e').$$

L'idea: e precede e' se e solo se tutto il passato di e sta nel passato di e', e su ogni processo questo vuol dire che e' conosce almeno tanti eventi quanti ne conosce e. Per esempio $VC(e_2^1) = (0, 1, 0) < (5, 1, 6) = VC(e_3^6)$, quindi $e_2^1 \to e_3^6$.

Due eventi sono concorrenti? Sì, se e solo se i loro vector clock sono incomparabili. Per esempio $VC(e_2^2) = (1, 2, 4)$ e $VC(e_3^6) = (5, 1, 6)$ sono incomparabili, quindi e_2^2 ed e_3^6 sono concorrenti.

Se si sa su quali processi stanno i due eventi, basta guardare una sola componente. Sia e un evento di p_i, il k-esimo, cioè $VC(e)[i] = k$, ed e' un evento di un altro processo. Allora e precede e' se e solo se e' conosce almeno i primi k eventi di p_i:

$$e \to e' \iff VC(e')[i] \geq VC(e)[i].$$

Per esempio e_1^4 è il quarto evento di p_1, e $VC(e_2^3)[1] = 4$: e_2^3 conosce i primi quattro eventi di p_1, quindi $e_1^4 \to e_2^3$. Due eventi sono concorrenti quando nessuno dei due è tra quelli che l'altro conosce: e_2^2 è il secondo evento di p_2, ma e_3^6 conosce un solo evento di p_2; e_3^6 è il sesto evento di p_3, ma e_2^2 ne conosce solo quattro.

Un taglio è consistente? Basta guardare la frontiera, cioè l'ultimo evento di ogni processo nel taglio. Il taglio $(c_1, \ldots, c_n)$ è consistente se e solo se nessuno di questi eventi conosce, di un processo, più eventi di quanti il taglio ne contenga:

$$VC(e_j^{c_j})[i] \leq c_i \qquad \text{per ogni coppia di processi } p_i,\, p_j,$$

considerando solo i processi p_j che hanno almeno un evento nel taglio. Se invece $e_j^{c_j}$ conosce più eventi di p_i, dipende da un evento di p_i rimasto fuori. Per esempio, nel taglio $C' = (3, 2, 6)$ della Figura 2.1 l'ultimo evento di p_3, e_3^6, ha vector clock $(5, 1, 6)$: conosce cinque eventi di p_1, ma il taglio ne contiene tre, quindi C' è inconsistente. Eventi come e_1^3 ed e_3^6, che non possono stare sulla stessa frontiera di un taglio consistente, si dicono **inconsistenti a coppie** (*pairwise inconsistent*). Nel taglio $C = (5, 2, 4)$, invece, la condizione vale per ogni coppia.

Manca qualcosa in mezzo? I clock logici non avevano la gap detection (Sezione 3.5): guardando due timestamp non si può capire se esiste un evento «in mezzo», cioè un **buco** (*gap*) tra i due. I vector clock ne danno una forma debole, la **weak gap detection**. Se e' conosce più eventi di un processo p_k di quanti ne conosca e, allora c'è almeno un evento di p_k nel passato di e' che non sta nel passato di e:

$$VC(e)[k] < VC(e')[k] \implies \text{esiste un evento } f \text{ di } p_k \text{ con } f \not\to e \text{ e } f \to e'.$$

Per esempio $VC(e_1^2) = (2, 1, 0)$ e $VC(e_2^3) = (4, 3, 4)$. Nella terza componente $0 < 4$: e_2^3 conosce quattro eventi di p_3 ed e_1^2 nessuno, e infatti e_3^1 sta nel passato di e_2^3 ma non in quello di e_1^2.

La proprietà è debole perché in generale non dice che l'evento trovato sta *tra* i due: e_3^1 non viene dopo e_1^2, perché $e_1^2 \not\to e_3^1$. Lo dice però quando p_k è il processo di e. In quel caso l'evento trovato è un evento di p_k successivo a e, quindi $e \to f \to e'$: f sta proprio nel buco. Per esempio, nella prima componente $VC(e_1^2)[1] = 2 < 4 = VC(e_2^3)[1]$: e_2^3 conosce due eventi di p_1 successivi a e_1^2, cioè e_1^3 ed e_1^4, che stanno tra i due. È questo il caso che userà il monitor per accorgersi che gli manca una notifica (Sezione 4.6).

4.6 Consegna causale con i vector clock

Torniamo al monitor. I processi gli inviano una notifica per ogni evento rilevante, e ogni notifica porta come timestamp il vector clock dell'evento notificato. C'è un accorgimento: i processi incrementano la propria componente solo negli eventi che notificano, oppure, che è lo stesso, notificano tutti i loro eventi. Il motivo è che il monitor vede soltanto le notifiche: con questo accorgimento le componenti dei vector clock contano notifiche, e il monitor le può confrontare con quelle che ha ricevuto.

Per una notifica m inviata da p_j, le componenti del timestamp $TS(m)$ dicono allora:

- $TS(m)[j]$, la componente del mittente: la posizione di m tra le notifiche di p_j (m è la prima, la seconda, ...);
- $TS(m)[k]$, per ogni altro processo p_k: quante notifiche di p_k precedono causalmente m, cioè quante ne conosceva p_j quando ha inviato m.

Il monitor, dal canto suo, tiene da parte le notifiche ricevute ma non ancora consegnate, e un contatore per ogni processo, $D[1], \ldots, D[n]$, inizialmente a zero: $D[k]$ è il numero di notifiche di p_k che ha già consegnato.

Quando può consegnare una notifica m di p_j? Quando è sicuro che non esiste nessun'altra notifica, già arrivata o ancora in viaggio, che debba venire prima di m. Per esserne sicuro deve rispondere sì a due domande.

1. *È la prossima notifica di p_j?* Le notifiche di p_j possono arrivare in disordine, e vanno consegnate nell'ordine in cui sono state inviate: se m è la terza notifica di p_j, il monitor deve averne già consegnate esattamente due.
2. *Ha già consegnato tutto ciò che p_j conosceva?* Quando ha inviato m, p_j conosceva un certo numero di notifiche di ogni altro processo p_k, e tutte precedono causalmente m: il monitor deve averle già consegnate. Se non le ha consegnate tutte, ne manca almeno una che viene prima di m: è la weak gap detection, con il buco su p_k.

Se una delle due risposte è no, la notifica resta in attesa, e il monitor la riesamina ogni volta che consegna qualcos'altro.

Definizione (regola DR3, consegna causale). Il monitor consegna la notifica m di p_j non appena

$$D[j] = TS(m)[j] - 1 \qquad \text{e} \qquad D[k] \geq TS(m)[k] \ \text{ per ogni } k \neq j,$$

e dopo la consegna aggiorna il contatore di p_j: $D[j] := TS(m)[j]$.

Esempio (la regola DR3 al lavoro). Nella Figura 4.4 ci sono due processi, p_1 e p_2, e il monitor p_0. I vector clock hanno due componenti, una per processo, e si incrementano solo negli eventi notificati.

- p_1 esegue un evento notificato, con vector clock $(1, 0)$, in cui invia un messaggio a p_2; la sua notifica è m, la prima di p_1.

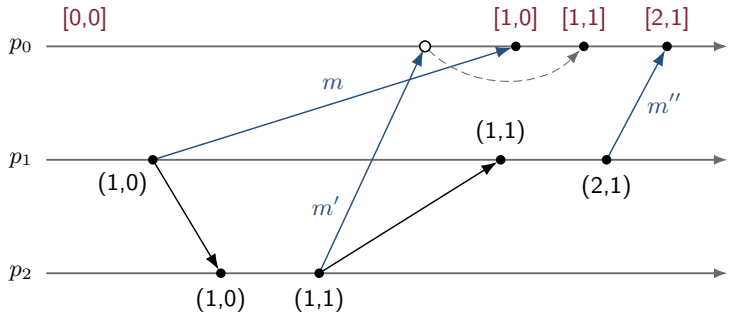


Figura 4.4. La regola DR3 al lavoro. Su p_1 e p_2 i vector clock, su p_0 (in rosso) i valori di D; in blu le notifiche. m' arriva prima di m (cerchio vuoto) e viene consegnata solo dopo di essa.

- p_2 riceve il messaggio. L'evento non è notificato, quindi il vector clock diventa $\max\{(0,0),(1,0)\} = (1,0)$ senza incremento. Poi p_2 esegue un evento notificato, con vector clock $(1,1)$, in cui invia un messaggio a p_1; la sua notifica è m', la prima di p_2.
- m' arriva a p_0 per prima, con $TS(m') = (1,1)$ e $D = [0,0]$. È la prossima notifica di p_2 ($D[2] = 0 = 1-1$), ma p_2 conosceva una notifica di p_1 che il monitor non ha ancora consegnato ($D[1] = 0 < 1$): m' resta in attesa.
- Arriva m, con $TS(m) = (1,0)$: è la prossima notifica di p_1 ($D[1] = 0 = 1-1$) e p_1 non conosceva notifiche di p_2 ($D[2] = 0 \geq 0$), quindi m viene consegnata e D diventa $[1,0]$. Ora anche m' supera la seconda domanda ($D[1] = 1 \geq 1$): viene consegnata e D diventa $[1,1]$.
- p_1 riceve il messaggio di p_2 (vector clock $(1,1)$) e poi esegue un evento notificato, $(2,1)$, con notifica m''. Quando m'' arriva è la prossima notifica di p_1 ($D[1] = 1 = 2-1$) e il monitor ha già consegnato la notifica di p_2 che p_1 conosceva ($D[2] = 1 \geq 1$): viene consegnata subito, e D diventa $[2,1]$.

Con la regola DR3 si ha una soluzione completa di GPE con monitoraggio passivo in un sistema asincrono. I processi notificano a p_0 gli eventi rilevanti; p_0 consegna le notifiche con una regola di consegna causale e ottiene un'osservazione che corrisponde a un run consistente; Φ si può valutare in qualunque stato globale del run, perché sono tutti consistenti.

La stessa regola si può usare in qualunque processo, non solo nel monitor: se tutti comunicano con *broadcast*, cioè inviando ogni messaggio a tutti, DR3 dà il *causal broadcast*, una primitiva molto usata per costruire applicazioni distribuite.

Canali nascosti. Le conclusioni tratte dalla precedenza causale valgono solo in un **sistema chiuso**, in cui tutta la comunicazione passa per i canali del sistema. Un **canale nascosto** (*hidden channel*), esterno al sistema, la relazione $\to$ non lo vede. Per esempio p_1 rileva la rottura di un tubo del vapore e la notifica a

> p_3, che controlla il riscaldamento; poco dopo p_2, più a valle, rileva un calo di pressione e notifica anche quello. Per il sistema le due notifiche sono concorrenti: p_3 può vedere prima il calo di pressione, aumentare il calore, e concludere a torto che il tubo si è rotto per il calore. Il legame tra rottura e calo di pressione passa per il tubo, fuori dal sistema; in casi così bisogna ordinare le osservazioni con il tempo reale.

Capitolo 5

Snapshot distribuiti

Si torna alla strategia attiva del capitolo 2: il monitor «fotografa» gli stati dei processi e li mette insieme. Lì il risultato poteva essere inconsistente; qui si costruisce un protocollo di **snapshot** (istantanea) che produce solo stati globali consistenti, senza fermare la computazione. È il protocollo di Chandy e Lamport, e ci si arriva in tre passi: un protocollo semplice per un sistema sincrono, la sua trasformazione in un protocollo per sistemi asincroni, e infine le proprietà dello stato che costruisce. In tutto il capitolo si assume che i canali siano FIFO.

5.1 Stati dei canali

Uno snapshot è una fotografia del sistema: gli stati locali dei processi e i messaggi che in quel momento sono in viaggio. Le difficoltà sono due: gli stati dei processi non si possono registrare tutti nello stesso istante, e i messaggi in viaggio non li vede nessun processo.

> **Definizione (stato di un canale).** Lo **stato del canale** da p_i a p_j, indicato con $\chi_{i,j}$ (con la lettera greca *chi*), è l'insieme dei messaggi in viaggio su quel canale: quelli che p_i ha inviato a p_j e che p_j non ha ancora ricevuto.

Come si è visto nel capitolo 2, lo stato di un canale si potrebbe ricavare dagli stati locali: i messaggi in viaggio da p_i a p_j sono quelli che p_i ha inviato a p_j (registrati nel suo stato σ_i) tranne quelli che p_j ha già ricevuto (registrati in σ_j). Registrarlo a parte però è comodo, perché gli stati locali restano più semplici e i predicati si scrivono più facilmente.

Per ogni processo p_i si distinguono i **canali entranti**, su cui p_i riceve, e i **canali uscenti**, su cui invia; il TR indica con IN_i e OUT_i gli insiemi dei processi all'altro capo di questi canali. In uno snapshot ogni processo registra il proprio stato locale e lo stato di ciascuno dei suoi canali entranti: così ogni canale viene registrato una volta sola, dal processo che riceve su di esso.

5.2 Primo protocollo: un orologio reale

Si parte, come al solito, da ipotesi forti: tutti i processi leggono lo stesso orologio reale RC, i ritardi dei messaggi sono limitati da un valore noto, e lo sono anche le velocità relative dei processi. L'idea è la più naturale: tutti registrano il proprio stato nello stesso istante, come una foto di gruppo scattata da tutti insieme. Il monitor p_0 sceglie questo istante, che il TR chiama t_{ss} (*time* dello *snapshot*), abbastanza avanti nel tempo da essere sicuro che il suo annuncio arrivi a tutti prima di t_{ss}: i limiti sui ritardi gli permettono di calcolarlo. Per poter registrare gli stati dei canali, ogni messaggio porta come timestamp l'istante in cui è stato inviato.

Definizione (Protocollo 1, con orologio reale).

1. p_0 invia a tutti i processi il messaggio «take snapshot at t_{ss}».
2. Quando l'orologio segna t_{ss}, ogni processo p_i registra il proprio stato locale σ_i, invia un messaggio vuoto (senza contenuto, con timestamp t_{ss}) su ogni canale uscente e comincia a registrare i messaggi che riceve su ogni canale entrante. Registrazione e invio dei messaggi vuoti avvengono prima di qualunque altro evento della computazione.
3. La prima volta che p_i riceve da p_j un messaggio con timestamp maggiore o uguale a t_{ss}, smette di registrare quel canale: $\chi_{j,i}$ è l'insieme dei messaggi registrati.

Perché la registrazione di un canale si chiude così? Sul canale da p_j a p_i i messaggi arrivano nell'ordine in cui sono partiti, perché il canale è FIFO. Quando p_i riceve da p_j il primo messaggio con timestamp almeno t_{ss}, cioè inviato dopo lo snapshot, sa che tutti i messaggi inviati da p_j prima di t_{ss} sono già arrivati. I messaggi registrati, arrivati dopo t_{ss} ma inviati prima, sono quindi esattamente quelli che nell'istante dello snapshot erano in viaggio. I messaggi vuoti servono a garantire che la registrazione finisca: senza di loro, un canale su cui p_j non invia più nulla resterebbe registrato per sempre, mentre così su ogni canale entrante arriva prima o poi un messaggio con timestamp almeno t_{ss}.

Lo stato costruito è consistente, ed è anzi uno stato che si è verificato davvero: è una foto scattata da tutti nello stesso istante. Conviene però vedere quale proprietà dell'orologio garantisce la consistenza. Il taglio dello snapshot contiene esattamente gli eventi avvenuti prima di t_{ss}. Se un evento e è nel taglio ed e' è una sua causa, per la clock condition e' è avvenuto ancora prima di e, quindi prima di t_{ss}, ed è anch'esso nel taglio; in formula, $RC(e') < RC(e) < t_{ss}$. Il taglio contiene le cause di ogni suo evento, cioè è consistente. Dell'orologio reale serve soltanto, ancora una volta, la clock condition.

5.3 Dall'orologio reale al protocollo di Chandy–Lamport

Il Protocollo 1 usa dell'orologio reale soltanto la clock condition, e i clock logici la soddisfano anche in un sistema asincrono. Il protocollo di Chandy–Lamport si

ottiene da qui in tre passi; la Tabella 5.1 riassume come cambiano, a ogni passo, le regole per registrare lo stato e per chiudere la registrazione di un canale.

Primo passo: i clock logici (Protocollo 2). Si sostituisce l'orologio reale con i clock logici, con due accorgimenti. Un clock logico cambia solo quando avvengono eventi e può saltare dei valori (nella Figura 3.1 quello di p_3 passa da 5 a 7): servono alcune regole tecniche, che qui si omettono. Inoltre p_0 non può più scegliere un istante «abbastanza avanti», perché in un sistema asincrono non sa quanto andranno avanti i clock degli altri prima che arrivi il suo annuncio. Si usa allora un valore ω così grande che nessun clock lo raggiunge con i normali eventi della computazione. p_0 porta il proprio clock a ω, registra il proprio stato e invia i messaggi vuoti con timestamp ω, che a catena portano a ω i clock degli altri: è il protocollo stesso, e non lo scorrere del tempo, a decidere quando ognuno registra. Resta un difetto: anche l'esistenza di un valore così grande richiede di limitare velocità e ritardi.

Secondo passo: niente annuncio. Nel Protocollo 2, a far registrare un processo è il primo messaggio vuoto che riceve, con timestamp ω; l'annuncio «take snapshot at ω» non serve a niente. Lo si elimina, e il messaggio vuoto diventa un messaggio speciale, «take snapshot», detto anche **marker** (segnalino), per non confonderlo con altri messaggi vuoti.

Terzo passo: niente timestamp. Con canali FIFO, il marker è il primo messaggio che un processo invia su ogni canale dopo aver registrato il proprio stato. Su ogni canale, quindi, il marker *separa* i messaggi inviati prima della registrazione del mittente, che arrivano prima di lui, da quelli inviati dopo, che arrivano dopo. È l'idea su cui si regge tutto il protocollo. I timestamp non servono più a distinguere i messaggi, e con loro spariscono i clock logici e il valore ω. Resta il protocollo di Chandy–Lamport, che funziona in un sistema asincrono con la sola ipotesi dei canali FIFO.

Tabella 5.1. I tre protocolli di snapshot a confronto.

	Protocollo 1	Protocollo 2	Chandy–Lamport
Quando p_i registra il proprio stato	quando l'orologio reale segna t_{ss}	quando il suo clock logico raggiunge ω	alla prima ricezione di un marker
Quando smette di registrare il canale da p_j	al primo messaggio di p_j con timestamp $\geq t_{ss}$	al primo messaggio di p_j con timestamp $\geq \omega$	quando arriva il marker di p_j
Ipotesi, oltre ai canali FIFO	orologio comune; ritardi e velocità limitati	un valore ω che nessun clock raggiunge da solo	nessuna

5.4 Il protocollo di Chandy–Lamport

Definizione (protocollo di Chandy–Lamport).

1. p_0 avvia il protocollo inviando un marker a sé stesso.
2. Alla *prima* ricezione di un marker, p_i registra il proprio stato locale σ_i e invia un marker su ogni canale uscente, senza eseguire eventi della computazione tra le due cose. Il canale da cui è arrivato questo primo marker si registra vuoto; su ognuno degli altri canali entranti p_i comincia a registrare i messaggi che arrivano.
3. Ogni volta che riceve un marker *successivo*, su un altro canale entrante, p_i smette di registrare quel canale: il suo stato è l'insieme dei messaggi registrati.

Il TR scrive gli stessi passi con due nomi: p_f (da *first*) è il processo da cui p_i riceve il primo marker, e lo stato del suo canale è $\chi_{f,i} = \emptyset$; p_s indica uno qualunque dei processi da cui arriva un marker successivo, e $\chi_{s,i}$ è l'insieme dei messaggi registrati sul canale da p_s. Indichiamo inoltre con e_i^* l'evento in cui p_i riceve il primo marker e registra il proprio stato. Non è un evento della computazione, ma serve a dire quali eventi di p_i stanno prima della registrazione e quali dopo; nelle figure è un quadratino.

p_0 manda il primo marker a sé stesso solo per seguire anche lui la regola del passo 2: registra il proprio stato e manda il marker a tutti i vicini. Un processo inoltra il marker soltanto alla prima ricezione, e la rete è fortemente connessa, quindi il marker raggiunge tutti e attraversa ogni canale esattamente una volta. Quando p_i ha ricevuto il marker da tutti i canali entranti, ha registrato il proprio stato e quelli di tutti i suoi canali entranti: la sua parte del protocollo è finita, e resta solo da inviare i risultati a p_0, un dettaglio che qui si trascura.

Perché i canali vengono registrati così? Lo stato del canale da p_j a p_i deve contenere i messaggi inviati prima della registrazione di p_j e ricevuti dopo quella di p_i. Poiché il marker di p_j separa i messaggi inviati prima della registrazione da quelli inviati dopo (Sezione 5.3), sono proprio quelli che p_i riceve su quel canale tra la propria registrazione e l'arrivo del marker. Sul canale da cui arriva il primo marker questo intervallo è vuoto, e per questo il canale si registra vuoto.

Esempio (il protocollo passo passo). Nella Figura 5.1 il protocollo viene eseguito sulla computazione di due processi della Figura 2.3.

- p_0 avvia il protocollo: registra il proprio stato e invia il marker a p_1 e a p_2.
- p_1 riceve il marker di p_0 dopo e_1^2 (e_1^*): registra σ_1^2, inoltra il marker a p_0 e a p_2, registra vuoto il canale da p_0 ($\chi_{0,1} = \emptyset$) e comincia a registrare il canale da p_2.
- Il marker di p_0 arriva a p_2 solo dopo e_2^3 (e_2^*): p_2 registra σ_2^3, inoltra il marker a p_0 e a p_1, pone $\chi_{0,2} = \emptyset$ e comincia a registrare il canale da p_1.

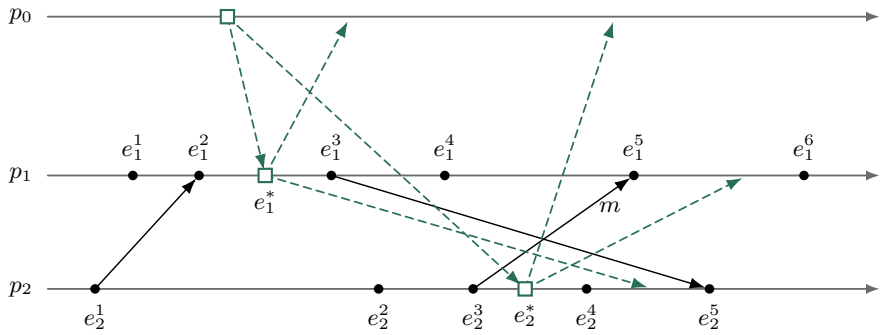


Figura 5.1. Il protocollo di Chandy–Lamport sulla computazione della Figura 2.3. Tratteggiati i marker; i quadratini segnano le registrazioni (e_1^* ed e_2^*).

- Il messaggio m, inviato da p_2 in e_2^3, cioè prima della sua registrazione, arriva a p_1 in e_1^5, dopo la registrazione di p_1 ma prima del marker di p_2: viene registrato. Quando poi arriva il marker di p_2, $\chi_{2,1} = \{m\}$.
- Sul canale da p_1 a p_2, il messaggio inviato in e_1^3 è partito dopo il marker di p_1 e arriva dopo di lui: quando il marker arriva, p_2 non ha registrato nulla su quel canale, e $\chi_{1,2} = \emptyset$.

Lo stato costruito è Σ^{23}, perché p_1 si è registrato dopo due eventi e p_2 dopo tre. Gli stati dei canali sono $\chi_{1,2} = \emptyset$ e $\chi_{2,1} = \{m\}$. Lo stato di p_0 non interessa: p_0 non partecipa alla computazione e serve solo ad avviare il protocollo. Lo stato costruito è consistente: l'unico messaggio ricevuto dentro il taglio, quello inviato in e_2^1, è stato inviato dentro il taglio; e m, inviato dentro e ricevuto fuori, compare correttamente tra i messaggi in viaggio.

5.5 Perché funziona, e perché servono i canali FIFO

Lo stato costruito dal protocollo è sempre consistente. Un taglio è inconsistente solo se contiene un messaggio «dal futuro», ricevuto dentro il taglio ma inviato fuori (capitolo 2). Ma un messaggio che p_j invia a p_k dopo essersi registrato, cioè fuori dal taglio, viaggia dietro al marker di p_j (Sezione 5.3): quando arriva, p_k si è già registrato, e anche la ricezione è fuori dal taglio. Nessun messaggio arriva dal futuro, e il taglio è consistente. È qui che servono i canali FIFO: se il messaggio potesse sorpassare il marker, p_k potrebbe riceverlo prima di registrarsi, e lo stato costruito conterrebbe una ricezione senza il suo invio. Il protocollo si può estendere in molti modi, per esempio per farlo funzionare senza canali FIFO o per ridurre il numero di messaggi.

5.6 Proprietà dello snapshot

Per capire dove si colloca lo stato costruito rispetto all'esecuzione reale servono tre stati globali. Hanno i nomi del TR:

- Σ^a (a come avvio): lo stato in cui si trova il sistema, nel run reale, quando p_0 avvia il protocollo;
- Σ^f (f come fine): lo stato in cui si trova quando il protocollo termina, cioè quando arriva l'ultimo marker;
- Σ^s (s come snapshot): lo stato costruito dal protocollo.

Σ^a e Σ^f sono stati attraversati davvero. Σ^s è consistente, ma non è detto che il sistema ci sia mai passato.

Esempio (uno snapshot mai accaduto). Nell'esecuzione della Figura 5.1 il run seguito davvero dai processi è

$$e_2^1\, e_1^1\, e_1^2\, e_1^3\, e_2^2\, e_1^4\, e_2^3\, e_2^4\, e_1^5\, e_2^5\, e_1^6,$$

cioè, in termini di stati globali,

$$\Sigma^{00}\, \Sigma^{01}\, \Sigma^{11}\, \Sigma^{21}\, \Sigma^{31}\, \Sigma^{32}\, \Sigma^{42}\, \Sigma^{43}\, \Sigma^{44}\, \Sigma^{54}\, \Sigma^{55}\, \Sigma^{65}.$$

Il protocollo parte in $\Sigma^a = \Sigma^{21}$, finisce in $\Sigma^f = \Sigma^{55}$ e costruisce $\Sigma^s = \Sigma^{23}$, che nel run non compare: quando p_1 si è registrato, p_2 aveva eseguito un solo evento, e quando si è registrato p_2, p_1 era già arrivato a e_1^4.

Lo stato costruito non è però uno stato consistente qualunque: sta sempre tra l'avvio e la fine del protocollo,

$$\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f,$$

cioè esiste un run consistente che passa prima per Σ^a, poi per Σ^s e infine per Σ^f. Il run reale magari non passa per Σ^s, ma un altro run consistente sì: uno dei run che il sistema avrebbe potuto seguire.

Perché. L'argomento del TR divide gli eventi di ogni processo in due gruppi. Un evento di p_i è **pre-registrazione** (*prerecording*) se avviene prima di e_i^*, cioè prima che p_i registri il proprio stato, ed è **post-registrazione** (*postrecording*) se avviene dopo. Lo stato costruito Σ^s è proprio lo stato in cui si arriva eseguendo tutti gli eventi pre-registrazione e nessun altro. Nell'esempio sono pre-registrazione e_1^1, e_1^2 e e_2^1, e_2^2, e_2^3; tutti gli altri sono post-registrazione.

Il punto chiave è che un evento post-registrazione non può precedere causalmente un evento pre-registrazione, per lo stesso argomento della Sezione 5.5: un messaggio inviato dopo la registrazione del mittente arriva dopo il marker, quindi viene ricevuto dopo la registrazione del destinatario. Allora nel run reale si possono spostare tutti gli eventi pre-registrazione prima di quelli post-registrazione, lasciando invariato l'ordine dentro ciascun gruppo, senza mai mettere un effetto prima della sua causa. Il run che si ottiene è consistente e passa per i tre stati:

- per Σ^a, perché gli eventi avvenuti prima dell'avvio sono tutti pre-registrazione (nessuno si registra prima che il protocollo parta) e restano in testa, nello stesso ordine;

- per Σ^s, subito dopo l'ultimo evento pre-registrazione;
- per Σ^f, perché gli eventi avvenuti dopo la fine sono tutti post-registrazione e restano in coda.

Nell'esempio, nel run reale gli eventi post-registrazione e_1^3 ed e_1^4 vengono prima degli eventi pre-registrazione e_2^2 ed e_2^3. Spostando questi ultimi in avanti si ottiene il run

$$e_2^1\, e_1^1\, e_1^2\, e_2^2\, e_2^3 \mid e_1^3\, e_1^4\, e_2^4\, e_1^5\, e_2^5\, e_1^6,$$

dove la sbarra separa i due gruppi. Passa per Σ^{21}, poi per Σ^{22} e Σ^{23}, poi per Σ^{33} e Σ^{43}, e da lì prosegue come il run reale fino a Σ^{55}: è il cammino verde della Figura 5.2.

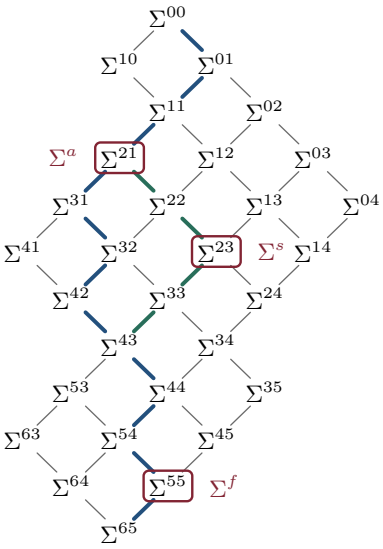


Figura 5.2. Lo snapshot nel reticolo della Figura 2.3. Il run reale (blu) passa per l'avvio $\Sigma^a = \Sigma^{21}$ e la fine $\Sigma^f = \Sigma^{55}$, ma non per lo stato costruito $\Sigma^s = \Sigma^{23}$. Il run riordinato (verde) se ne discosta solo tra Σ^{21} e Σ^{43}, e passa per tutti e tre.

A che cosa serve. Se lo stato costruito può non essere mai esistito, e in ogni caso descrive il passato, a che cosa serve? A rilevare le proprietà **stabili**, quelle che, una volta diventate vere, restano vere: per esempio il deadlock o la terminazione. Per una proprietà così, da $\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f$ segue che, se vale nello stato costruito, vale anche adesso; e se non vale, non valeva nemmeno quando il protocollo è partito. Le proprietà stabili si riprenderanno più avanti.